

Refactoring Candidates Not Directly Accepted

14 Cases, with the Real Code

CloneDeMocker attempted an automated refactor for every duplicated Mock setup (Mock Clone Instance, MCI) detected in Apache Dubbo. 14 candidates did not come out of that as a direct success.

This report shows what each one actually looked like, and why.

Overview

The 14 cases split first by whether the model itself raised an objection:

Group 1: AI Flagged as Disputed -- 11 cases

Before touching any code, the model judged whether this candidate fit its one allowed move (extract the duplicated Mock setup into a shared helper and wire it in). If it decided the move was unsafe or out of scope, it flagged the candidate as disputed, explained why, and produced no patch. Nothing here ever reached compile or test -- it isn't a "failure", it's the model's own call.

Subtype	Pattern	N
A. Dead config	Mock is never wired into the object under test	3
B. Semantic divergence	Cases deliberately configure the same mock differently	4
C. Tooling constraint	Fix needs a new shared file; tool can't create one	2 (1 overlaps B)
D. Needs runtime proof	Static reading can't confirm the change is safe	1
E. Full-file protocol limit	File too large to safely return whole	2

Group 2: Executed, Then Actually Failed -- 3 cases

Here the model raised no objection: it produced a patch and the patch was run -- compiled, tested, format-checked. Only after execution did it turn out not to hold up. These are real failures, each for a different reason:

Subtype	Pattern	N
F. Goal partly met	Compiles and passes tests, but only some duplication was removed	2
G. Style-check conflict	Valid, compilable code that trips the target project's formatter	1

Blue boxes below = "AI OBJECTION" (Group 1). Orange boxes = "FAILURE" (Group 2).

Contents

A. Dead config -- mock never wired into the object under test

Case 4 org.apache.dubbo.metadata.MetadataService::1

Case 5 CompositeConfiguration::1 and ConfigManager::1

B. Semantic divergence -- deliberately different configs

Case 2 java.net.InetAddress::2

Case 1 org.apache.dubbo.registry.nacos.NacosConnectionManager::1

Case 10 org.apache.dubbo.registry.NamingService::4

Case 3 org.apache.dubbo.registry.client.ServiceDiscovery::1

C. Tooling constraint -- needs a new shared file

Case 8 org.apache.dubbo.registry.NotifyListener::2

D. Needs runtime proof

Case 7 org.apache.dubbo.rpc.Invoker::2

E. Full-file protocol limit

Case 9 ExtensionLoader<RegistryProtocolListener>::1

Case 11 org.apache.dubbo.common.logger.EventListener::1

F. Goal partly met

Case 12 org.apache.dubbo.rpc.Invoker::3

Case 13 Directory<DemoService>::1

G. Style-check conflict

Case 14 org.apache.dubbo.rpc.cluster.Cluster::1

A. Dead config -- mock never wired into the object under test

The mock is created and stubbed, but never passed into any constructor, setter, or method argument of the object actually being tested, so the stub can't influence anything. Extracting it into a helper would just relocate dead code, not the encapsulate-and-integrate refactor the tool targets -- so the model declined.

Case 4 `org.apache.dubbo.metadata.MetadataService::1`

`ServiceInstancesChangedListenerTest.java`

```
static MetadataService metadataService;  
...  
metadataService = Mockito.mock(MetadataService.class); // in @BeforeAll setUp()
```

The only method that touches it:

```
@Test  
@Order(12)  
public void testInstanceWithoutRevision() {  
    Set<String> serviceNames = new HashSet<>();  
    serviceNames.add("app1");  
    ServiceDiscovery serviceDiscovery = Mockito.mock(ServiceDiscovery.class);  
    listener = new ServiceInstancesChangedListener(serviceNames, serviceDiscovery);  
    ServiceInstancesChangedListener spyListener = Mockito.spy(listener);  
    Mockito.doReturn(null).when(metadataService).getMetadataInfo(eq(null));  
    ServiceInstancesChangedEvent event =  
        new ServiceInstancesChangedEvent("app1", app1InstancesWithNoRevision);  
    spyListener.onEvent(event);  
    assertTrue(true);  
}
```

AI OBJECTION

"the duplicated stubbing targets static metadataService mocks that are not injected into, referenced by, or otherwise reachable from the locally created ServiceDiscovery and ServiceInstancesChangedListener in either test method."

The listener's constructor only receives serviceDiscovery -- metadataService is unreachable dead code.
Correct call: this needs a dead-code flag, not a refactor.

Case 5 `CompositeConfiguration::1` and `ConfigManager::1`

`RegistryProtocolTest.java` -- both mocks live in the same 7 test methods; shown together since they're the same pattern

```

@Test
void testConsumerUrlWithoutProtocol() {
    ApplicationConfig applicationConfig = new ApplicationConfig();
    applicationConfig.setName("application1");
    ConfigManager configManager = mock(ConfigManager.class);
    when(configManager.getApplicationOrElseThrow()).thenReturn(applicationConfig);
    CompositeConfiguration compositeConfiguration = mock(CompositeConfiguration.class);
    when(compositeConfiguration.convert(Boolean.class, ENABLE_CONFIGURATION_LISTEN, true))
        .thenReturn(true);
    Map<String, String> parameters = new HashMap<>();
    parameters.put(INTERFACE_KEY, DemoService.class.getName());
    parameters.put("registry", "zookeeper");
    parameters.put("register", "false");
    parameters.put(REGISTER_IP_KEY, "172.23.236.180");
    Map<String, Object> attributes = new HashMap<>();
    ServiceConfigURL serviceConfigURL = new ServiceConfigURL(
        "registry", "127.0.0.1", 2181,
        "org.apache.dubbo.registry.RegistryService", parameters);
    Map<String, String> refer = new HashMap<>();
    attributes.put(REFER_KEY, refer);
    attributes.put("key1", "value1");
    URL url = serviceConfigURL.addAttributes(attributes);
    RegistryFactory registryFactory = mock(RegistryFactory.class);
    Registry registry = mock(Registry.class);
    RegistryProtocol registryProtocol = new RegistryProtocol();
    MigrationRuleListener migrationRuleListener = mock(MigrationRuleListener.class);
    List<RegistryProtocolListener> registryProtocolListeners = new ArrayList<>();
    registryProtocolListeners.add(migrationRuleListener);
    ModuleModel moduleModel =
        Mockito.spy(ApplicationModel.defaultModel().getDefaultModule());
    moduleModel.getApplicationModel().getApplicationConfigManager()
        .setApplication(new ApplicationConfig("application1"));
    ExtensionLoader<RegistryProtocolListener> extensionLoaderMock =
        mock(ExtensionLoader.class);
    Mockito.when(moduleModel.getExtensionLoader(RegistryProtocolListener.class))
        .thenReturn(extensionLoaderMock);
    Mockito.when(extensionLoaderMock.getActivateExtension(url, REGISTRY_PROTOCOL_LISTENER_KEY))
        .thenReturn(registryProtocolListeners);
    url = url.setScopeModel(moduleModel);
    when(registryFactory.getRegistry(registryProtocol.getRegistryUrl(url)))
        .thenReturn(registry);
    Cluster cluster = mock(Cluster.class);

    Invoker<?> invoker =
        registryProtocol.doRefer(cluster, registry, DemoService.class, url, parameters);
    Assertions.assertTrue(invoker instanceof MigrationInvoker);
    // ... assertions continue, none of them touch configManager or
    // compositeConfiguration -- only cluster/registry/url/parameters reach doRefer()
}

```

AI OBJECTION

"each CompositeConfiguration mock is created and stubbed but is never passed to RegistryProtocol, ModuleModel, ConfigManager, or any other collaborator ... [same wording, both mocks] ... never passed to RegistryProtocol, ModuleModel, ApplicationModel, or any other collaborator used by the test. Removing them would be dead-code cleanup rather than the requested two-step mock-clone refactoring."

Same dead-mock pattern, twice, in the same method, repeated across all 7 test methods in this file -- likely leftover test debt in the target project, not a detector false positive. Correct call both times.

B. Semantic divergence -- deliberately different configs

The mock looks structurally similar (same class, same method names) but each test method configures it with different values, deliberately, to exercise different branches. Merging into one shared setup would erase that distinction, so the model declined.

Case 2 java.net.InetAddress::2

NetUtilsTest.java

```
@Test
void testIsValidAddress() {
    assertFalse(NetUtils.isValidV4Address((InetAddress) null));
    InetAddress address = mock(InetAddress.class);
    when(address.isLoopbackAddress()).thenReturn(true);
    assertFalse(NetUtils.isValidV4Address(address));
    address = mock(InetAddress.class);
    when(address.getHostAddress()).thenReturn("localhost");
    assertFalse(NetUtils.isValidV4Address(address));
    address = mock(InetAddress.class);
    when(address.getHostAddress()).thenReturn("0.0.0.0");
    assertFalse(NetUtils.isValidV4Address(address));
    address = mock(InetAddress.class);
    when(address.getHostAddress()).thenReturn("127.0.0.1");
    assertFalse(NetUtils.isValidV4Address(address));
    address = mock(InetAddress.class);
    when(address.getHostAddress()).thenReturn("1.2.3.4");
    assertTrue(NetUtils.isValidV4Address(address)); // the one branch that returns true
}
```

AI OBJECTION

"The two clone instances require different InetAddress configurations: one requires isLoopbackAddress, getHostAddress, and isReachable stubs on the same mock, while the other intentionally creates separate mocks with selectively configured stubs for independent assertions."

Five reassignments of the same variable, five different stub combos, one designed to hit the single true-returning branch. Collapsing this into a shared setup would erase the branch coverage. Correct call.

Case 1 org.apache.dubbo.registry.nacos.NacosConnectionManager::1

NacosNamingServiceWrapperTest.java -- testSubscribe vs. testConcurrency

```
@Test
void testSubscribe() throws NacosException {
    NacosConnectionManager connectionManager = Mockito.mock(NacosConnectionManager.class);
    NamingService namingService = Mockito.mock(NamingService.class);
    Mockito.when(connectionManager.getNamingService()).thenReturn(namingService);
    NacosNamingServiceWrapper nacosNamingServiceWrapper =
        new NacosNamingServiceWrapper(connectionManager, 0, 0);
    EventListener eventListener = Mockito.mock(EventListener.class);
    nacosNamingServiceWrapper.subscribe("service_name", "test", eventListener);
    Mockito.verify(namingService, Mockito.times(1))
        .subscribe("service_name", "test", eventListener);
    // ... repeats subscribe/unsubscribe with times(2), times(1), times(0)
}
```

testConcurrency uses its own independent instance:

```

@Test
void testConcurrency() throws NacosException, InterruptedException {
    NacosConnectionManager connectionManager = Mockito.mock(NacosConnectionManager.class);
    CountDownLatch startLatch = new CountDownLatch(1);
    CountDownLatch stopLatch = new CountDownLatch(1);
    NamingService namingService = Mockito.mock(NamingService.class);
    Mockito.when(connectionManager.getNamingService()).thenReturn(namingService);
    // ... registers an instance, then two threads register/deregister
    // concurrently against this connectionManager while a stubbed
    // InstancesInfo.getInstances() sleeps to force interleaving
    new Thread(() -> { /* register, countDown */ }).start();
    new Thread(() -> { /* deregister */ }).start();
}

```

AI OBJECTION

"each mock is method-local and testConcurrency relies on its own independently configured connection manager during concurrent registration and deregistration."

testConcurrency is a real concurrency test; it can't share a mock lifecycle with testSubscribe. Correct call.

Case 10 org.apache.dubbo.registry.NamingService::4

NacosNamingServiceWrapperTest.java -- testSubscribeMultiManager

```

@Test
void testSubscribeMultiManager() throws NacosException {
    NacosConnectionManager connectionManager = Mockito.mock(NacosConnectionManager.class);
    NamingService namingService1 = Mockito.mock(NamingService.class);
    NamingService namingService2 = Mockito.mock(NamingService.class);
    NacosNamingServiceWrapper nacosNamingServiceWrapper =
        new NacosNamingServiceWrapper(connectionManager, 0, 0);
    EventListener eventListener = Mockito.mock(EventListener.class);
    Mockito.when(connectionManager.getNamingService()).thenReturn(namingService1);
    nacosNamingServiceWrapper.subscribe("service_name", "test", eventListener);
    Mockito.verify(namingService1, Mockito.times(1))
        .subscribe("service_name", "test", eventListener);
    Mockito.when(connectionManager.getNamingService()).thenReturn(namingService2);
    nacosNamingServiceWrapper.subscribe("service_name", "test", eventListener);
    // ... namingService1 and namingService2 represent the backend
    // before/after a connection-manager switch
}

```

AI OBJECTION

"The mocks are independently scoped to distinct connection-manager lifecycles (including anonymous manager overrides that capture different lists). Integrating them into a shared reusable mock would risk cross-test interaction and alter lifecycle behavior."

namingService1/namingService2 stand for two different backends before and after a switch; merging them into one reusable mock erases exactly the thing the test is checking. Correct call.

Case 3 org.apache.dubbo.registry.client.ServiceDiscovery::1

ServiceInstancesChangedListenerTest.java + ...WithoutEmptyProtectTest.java

```

@Test
@Order(11)
public void testRevisionFailureOnNotification() {
    // retry path: a notification with a bad revision should recover
    // once the metadata retries after 15s
    Set<String> serviceNames = new HashSet<>();
    serviceNames.add("app1");
    serviceNames.add("app2");
    listener = new ServiceInstancesChangedListener(serviceNames, serviceDiscovery);
    listener.onEvent(new ServiceInstancesChangedEvent("app1", app1Instances));
    when(serviceDiscovery.getRemoteMetadata(eq("222"), anyList()))
        .thenReturn(new Answer<MetadataInfo>() {
            @Override
            public MetadataInfo answer(InvocationOnMock invocationOnMock) {
                if (Thread.currentThread().getName()
                    .contains("Dubbo-framework-metadata-retry")) {
                    return metadataInfo_222;
                }
                return MetadataInfo.EMPTY;
            }
        });
    listener.onEvent(new ServiceInstancesChangedEvent("app2", app1FailedInstances2));
    // ... assert the retry-thread branch eventually recovers the addresses
}

```

AI OBJECTION

"Creating a dedicated shared fixture/helper would require adding a file not supplied for modification."

The stub's behavior checks the calling thread's name -- tied to a specific retry mechanism, and (see subtype C below) the shared-helper fix would need a new file the tool wasn't allowed to create.

C. Tooling constraint -- needs a new shared file

Different from the above: the model considered the refactor itself clear and safe, but at the time, the tool could only edit files already flagged by the detector, not create new ones -- and that's exactly what these two candidates needed.

Case 8 org.apache.dubbo.registry.NotifyListener::2

ServiceInstancesChangedListenerTest.java + ...WithoutEmptyProtectTest.java (22 sequences across 6 test methods)

```
// repeated in both test classes:
ServiceInstancesChangedListener listener =
    new ServiceInstancesChangedListener(serviceNames, serviceDiscovery);
when(notifyListener.getConsumerUrl()).thenReturn(consumerURL);
// ... 22 near-identical sequences total, split across two independently
// runnable test classes
```

AI OBJECTION

"A behavior-preserving extraction is technically possible for each individual mock creation/stubbing pair, but integrating the selected clones across both test classes would require either introducing a new shared test utility file or coupling one test class to a helper declared in the other. The supplied-file constraint prohibits adding the appropriate shared utility file, while cross-class coupling would make the independently executable test classes depend on an unrelated sibling test source file."

The model says outright: give it a new file and the fix is clear. Of all 14 cases, this is the one where loosening a tool constraint -- not changing the model's judgment -- would directly recover a success.

See also: Case 3, ServiceDiscovery::1 (same constraint) ->

D. Needs runtime proof

This one is different in tone: the model isn't saying the refactor shouldn't happen, it's saying it can't confirm from static reading alone that the refactor is safe.

Case 7 org.apache.dubbo.rpc.Invoker::2

MergeableClusterInvokerTest.java -- testInvokerToException

```
@Test
void testInvokerToException() {
    given(invocation.getMethodName()).willReturn("addMenu");
    given(invocation.getInvoker()).willReturn(firstInvoker);
    given(firstInvoker.getUrl()).willReturn(url.addParameter(GROUP_KEY, "first"));
    given(firstInvoker.getInterface()).willReturn(MenuService.class);
    given(firstInvoker.invoke(invocation)).willReturn(new AppResponse());
    given(firstInvoker.isAvailable()).willReturn(true);
    given(firstInvoker.invoke(invocation))
        .willThrow(new RpcException(RpcException.NETWORK_EXCEPTION)); // re-stub!
    given(secondInvoker.getUrl()).willReturn(url.addParameter(GROUP_KEY, "second"));
    given(secondInvoker.invoke(invocation)).willReturn(new AppResponse());
    given(secondInvoker.invoke(invocation))
        .willThrow(new RpcException(RpcException.NETWORK_EXCEPTION)); // re-stub!
    given(directory.list(invocation)).willReturn(List.of(firstInvoker, secondInvoker));
    mergeableClusterInvoker = new MergeableClusterInvoker<>(directory);
    try {
        mergeableClusterInvoker.invoke(invocation);
        fail();
    } catch (RpcException expected) {
        assertEquals(expected.getCode(), RpcException.NETWORK_EXCEPTION);
    }
}
```

AI OBJECTION

"the repeated stubbing is interleaved with later restubbing of invoke(invocation) in exception tests. Although a helper could encapsulate the common statements, safely validating Mockito stubbing precedence and all affected invocation behavior requires executing or inspecting the target implementation and test suite, which is not available in the supplied files."

invoke(invocation) is stubbed once, then re-stubbed to throw -- Mockito stubbing order matters here, and the model won't assert it's safe to restructure without being able to run it. A genuinely conservative, defensible edge case.

E. Full-file protocol limit

Not a judgment about the refactor at all -- a side effect of the I/O contract at the time, which required the model to return a complete modified file rather than a diff. On a large file, the model declined rather than risk silently corrupting the untouched parts.

Case 9 `ExtensionLoader<RegistryProtocolListener>::1`

RegistryProtocolTest.java -- the same large file as Case 5 (the extensionLoaderMock lines already shown there)

AI OBJECTION

"the requested result must include the complete modified source file, while the supplied file is too large to reproduce reliably without risking truncation or accidental behavioral changes."

Same file, same size problem as Case 5/6 -- a protocol issue, not a verdict on this specific mock.

Case 11 `org.apache.dubbo.common.logger.EventListener::1`

NacosNamingServiceWrapperTest.java -- same file and methods as Cases 1 and 10 (the EventListener mock creation line already shown there)

AI OBJECTION

"the supplied source content is incomplete for reliable reconstruction: the file is provided as a large inline excerpt without an independently addressable source artifact or checksum, making it unsafe to return a modified complete file while guaranteeing that all unchanged content is preserved byte-for-byte."

Same file-size ceiling again, this time framed around reconstruction safety rather than raw length.

F. Goal partly met

These two got no objection, ran, compiled, and passed every test. Only a line-count check against the original duplication found that only part of the MCI was actually deduplicated -- a real, if partial, failure.

Case 12 org.apache.dubbo.rpc.Invoker::3

ConnectivityValidationTest.java -- before

```
@Test
void testRetry() throws InterruptedException {
    Invocation invocation = new RpcInvocation();
    LoadBalance loadBalance = new RandomLoadBalance();
    invokerList.clear();
    invokerList.add(invoker1);
    invokerList.add(invoker2);
    directory.notify(invokerList);
    Assertions.assertEquals(2, directory.list(invocation).size());
    when(invoker1.isAvailable()).thenReturn(false);
    Assertions.assertEquals(invoker2, clusterInvoker.select(
        loadBalance, invocation, directory.list(invocation),
        Collections.singletonList(invoker2)));
    Assertions.assertEquals(1, directory.list(invocation).size());
    when(invoker1.isAvailable()).thenReturn(true);
    // ...
}
```

What the accepted patch did -- extracted a shared helper:

```
private void setInvokerAvailability(Invoker invoker, boolean available) {
    when(invoker.isAvailable()).thenReturn(available);
}

// call sites become:
setInvokerAvailability(invoker1, false);
setInvokerAvailability(invoker1, true);
```

FAILURE

"compileStatus=PASSED, testStatus=PASSED -- the harness accepted the run."

The `isAvailable()` stub -- the MCI's tracked duplication -- was correctly extracted everywhere. The automated goal-check nonetheless flagged this MCI as not fully resolved on its original run; compiling and passing tests is not the same bar as the goal-check's own line-occurrence count. Execution succeeded; the stricter goal metric did not.

Case 13 Directory<DemoService>:1

FailSafeClusterInvokerTest.java -- before

```
@Test
void testNoInvoke() {
    dic = mock(Directory.class);
    given(dic.getUrl()).willReturn(url);
    given(dic.getConsumerUrl()).willReturn(url);
    given(dic.list(invocation)).willReturn(null);
    given(dic.getInterface()).willReturn(DemoService.class);
    invocation.setMethodName("method1");
    resetInvokerToNoException();
    FailsafeClusterInvoker<DemoService> invoker = new FailsafeClusterInvoker<>(dic);
    try {
        invoker.invoke(invocation);
    } catch (RpcException e) {
        Assertions.assertTrue(e.getMessage().contains("No provider available"));
    }
}
```

After -- the accepted patch (real diff):

```
@Test
void testNoInvoke() {
    dic = mock(Directory.class);
    DirectoryTestFixture.stubMetadata(dic, url, DemoService.class);
    given(dic.list(invocation)).willReturn(null);
    // getUrl/getConsumerUrl/getInterface consolidated into stubMetadata();
    // "dic = mock(Directory.class);" itself is untouched, and still repeats
    // in BroadCastClusterInvokerTest's setUp()
    ...
}

final class DirectoryTestFixture {
    private DirectoryTestFixture() {}
    static <T> void stubMetadata(Directory<T> directory, URL url, Class<T> type) {
        given(directory.getUrl()).willReturn(url);
        given(directory.getConsumerUrl()).willReturn(url);
        given(directory.getInterface()).willReturn(type);
    }
}
```

FAILURE

"compileStatus=PASSED, testStatus=PASSED."

The three stub statements were correctly consolidated into a new DirectoryTestFixture helper -- a real, working Encapsulation-and-Integration move, and this run was even allowed to create the new file. But "dic = mock(Directory.class);" -- the actual duplicated creation line the MCI flagged -- was left untouched in both test classes. Genuinely partial: the helper is good, the job isn't finished.

G. Style-check conflict

Also no objection, also executed. The patch was valid, compilable Java. It failed a step after compilation: the target project's own formatter (Spotless) rejected the file's formatting -- unrelated to whether the refactor itself was sound.

Case 14 `org.apache.dubbo.rpc.cluster.Cluster::1`

RegistryProtocolTest.java -- the real build failure (compiled fine; Spotless didn't)

```
// before -- one line, matches the project's formatter:
moduleModel.getModel().getApplicationConfigManager()
    .setApplication(new ApplicationConfig("application1"));

// after -- the model's returned full file reformatted it to 4 lines:
moduleModel
    .getModel()
    .getApplicationConfigManager()
    .setApplication(new ApplicationConfig("application1"));
```

The actual build output:

```
[ERROR] Failed to execute goal com.diffplug.spotless:spotless-maven-plugin:
2.44.5:check (default) on project dubbo-registry-api:
The following files had format violations:
  src/test/java/.../RegistryProtocolTest.java
```

FAILURE

"Compiling 48 source files ... 0 errors, then: spotless:check FAILED."

The MCI itself was about deduplicating two identical "Cluster cluster = mock(Cluster.class);" lines in this file. The patch compiled cleanly; the failure came from the full-file rewrite protocol incidentally reformatting an unrelated method chain elsewhere in the same file, which then tripped the project's own style check. Not a judgment failure -- a side effect of returning whole files instead of diffs, later fixed by skipping Spotless during verification.